

EmbodiTTA: Resource-Efficient Test-Time Adaptation for Embodied Visual Systems

Xiao Ma, Young D. Kwon, *Member, IEEE*, and Dong Ma, *Member, IEEE*

Abstract—Deep neural networks (DNNs) are widely used for perception in embodied visual systems such as drones and AR glasses, where models must process streaming data under strict memory and energy budgets. However, once deployed in the physical world, these models often suffer from performance degradation in dynamic scenarios due to domain shifts. Although previous research has proposed test-time adaptation (TTA) to address this issue, which continuously adapts the deployed model *on every incoming batch of data before performing actual inference*, these approaches are impractical and inefficient from a system perspective, as they incur substantial memory and energy consumption and introduce high latency. In this work, we rethink the conventional paradigm of existing TTA and introduce a novel paradigm – on-demand TTA – which triggers adaptation only when a significant domain shift is detected. Then, we present EmbodiTTA, an efficient on-demand TTA framework for *accurate and efficient* TTA on embodied devices. EmbodiTTA comprises three techniques: 1) a lightweight domain shift detection mechanism to activate on-demand TTA only when needed, drastically reducing the overall computational overhead, 2) a source domain selection module to choose the appropriate starting point for adaptation, and 3) a decoupled Batch Normalization (BN) update scheme to reduce the memory overhead and achieve stable adaptation. We evaluated EmbodiTTA on the Jetson Orin Nano for both object recognition and semantic segmentation tasks. The results show that EmbodiTTA achieves the highest TTA accuracy across nearly all tasks and batch size settings—and is uniquely the only BN-based method effective at batch size 1—while reducing latency by up to $6.7\times$ and energy consumption by 47.2% compared to the baselines, with comparable peak memory usage.

Index Terms—Embodied AI, Mobile Vision, Domain Adaptation, Test-Time Adaptation

I. INTRODUCTION

Deep neural networks (DNNs) are becoming a core component of intelligent Internet-of-Things (IoT) and embodied AI systems [1], enabling egocentric (first-person) perception and on-device intelligence in resource-constrained platforms such as mobile robots, drones and AR/VR glasses, where models must process live sensory streams in real time. In these IoT-enabled embodied systems, visual perception often serves as the first step, providing scene understanding for downstream decision-making and interaction. However, as a

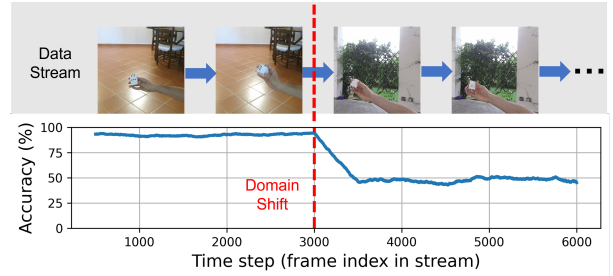


Figure 1. Impact of domain shift: indoor $\rightarrow$ outdoor.

data-driven technique, DNNs typically *achieve optimal performance only when training and testing data share the same distribution* [2], [3]. In the physical world, embodied systems inevitably encounter dynamic and unseen data distributions in their streaming test inputs, a phenomenon known as *domain shift*, caused by factors such as weather changes, sensor noise, and varying lighting conditions, which can lead to substantial performance degradation [4]. For example, a battery-powered delivery robot (or drone) must recognize obstacles in real time on an edge accelerator; when it moves from indoor to outdoor or from daylight to night, the same objects can look very different, causing recognition errors while the device still has to meet tight latency, memory, and energy budgets.

To illustrate the impact of domain shift, we conducted an experiment using CORE50 [5], a realistic continuous object recognition dataset for robots that contains indoor and outdoor sessions. As shown in Figure 1, we feed a continuous stream that transitions from an indoor session to an unseen outdoor session while keeping the object identity unchanged, and plot the rolling classification accuracy over time. The accuracy drops sharply from 94% to 49% after moving outdoor, even though the object identity remains unchanged. Since it is impractical to pretrain a model on all possible operating conditions, domain shift remains a key obstacle to robust embodied intelligence.

To address domain shift, **test-time adaptation (TTA)** [6] is an emerging solution. It adapts a pre-trained model *during deployment* using only unlabeled streaming inputs, allowing the perception model to adjust to previously unseen conditions on the fly. Representative TTA methods [7], [8], [9], [10] usually minimize the prediction entropy (an unsupervised loss function) on unlabeled target data, encouraging the model to make more confident predictions in the new domain. Such methods typically operate batch by batch (a chunk of streaming inputs) and alternate between two steps: (1) an *adaptation* step that updates model parameters via backpropagation, using the entropy-minimization objective, followed by (2) an *inference* step that produces predictions with the

*Dong Ma is the corresponding author.

Xiao Ma is with the Singapore Management University, Singapore, e-mail: xiaoma.2022@phdes.smu.edu.sg.

Young D. Kwon is with Samsung AI Center–Cambridge, United Kingdom, e-mail: yd.kwon@samsung.com

Dong Ma is with University of Cambridge, United Kingdom, e-mail: dm878@cam.ac.uk.

Copyright (c) 20xx IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

updated model. Repeating this procedure continuously adapts the model to each incoming batch, which is commonly referred to as **continual TTA (C-TTA)**.

However, embodied devices are typically constrained by tight memory and energy budgets, making direct deployment of C-TTA impractical for two reasons. First, C-TTA relies on backpropagation during inference, which incurs substantial memory overhead, and this cost becomes especially prohibitive as batch size increases. Second, in realistic streaming scenarios, domain shifts are often intermittent: long stable periods exhibit only minor distribution changes, so adapting every batch is unnecessary and offers marginal benefit. For example, in Figure 1, accuracy stays stable for a long duration and drops noticeably only after approximately 3000 frames with scene transition, suggesting that continuously adapting during the stable period is wasteful.

In this paper, we first introduce a practical and efficient paradigm, referred to as **on-demand TTA**, to address key deployment challenges in embodied systems. Unlike **continual TTA (C-TTA)**, which adapts the model on every incoming batch, on-demand TTA activates adaptation only when a significant domain shift is detected, namely when the shift leads to an application-defined and unacceptable performance drop. By avoiding unnecessary updates during stable periods, on-demand TTA substantially reduces memory and energy overhead. Moreover, when no significant shift is detected, the system runs in a lightweight inference-only mode, enabling fast streaming predictions and making it well-suited for latency-sensitive applications.

However, realizing on-demand TTA on embodied devices presents several challenges. **First**, on-demand TTA requires continuous monitoring of the data distribution for every incoming sample/batch for potential domain shift detection. However, *efficiently* quantifying the domain shift (or performance drop) without labels is challenging and remains underexplored in TTA literature. **Second**, differing from C-TTA, where the distribution of consecutive batches usually remains similar, on-demand TTA inherently deals with more severe shifts after a domain shift is detected. Existing C-TTA works have yet to address how to *effectively* adapt under more significant shifts. **Third**, a notable limitation of many C-TTA methods is their reliance on *large batch sizes* (therefore substantial memory consumption) to capture the new domain distribution [7], [8], which is impractical on embodied devices with limited onboard resources.

To tackle these challenges, we propose **EmbodiTTA**, a resource-efficient on-demand TTA framework designed for resource-constrained embodied devices. We drew three key insights from our observations and experimental studies to guide the design of EmbodiTTA. **First**, we observed that the entropy of model prediction can be used for detecting domain shifts. Based on this, we devised a *novel lightweight domain shift detection mechanism* using exponential moving average (EMA) entropy to detect the domain shift. **Second**, we observe that the choice of source domain substantially affects post-adaptation performance (e.g., adapting from *frost* to *snow* yields better results than adapting from *fog* to *snow*). Instead of always adapting from the immediately preceding domain

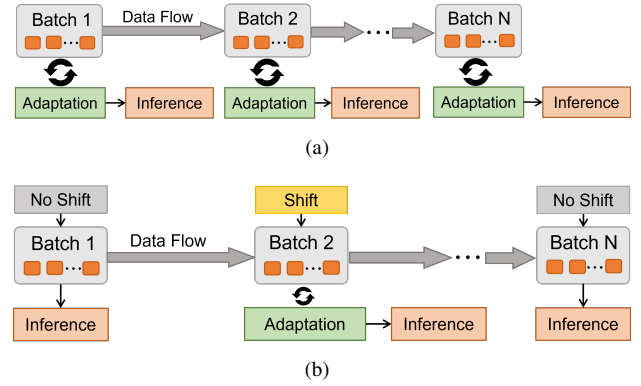


Figure 2. Illustration of (a) continual and (b) on-demand test-time adaptation.

as in C-TTA, we propose a *similar-domain selection* pipeline that identifies and selects the most relevant source domain for adaptation, leading to higher accuracy. **Third**, inspired by the insight that updating the Batch Normalization (BN) statistics and BN parameters consumes different amounts of memory and shows different sensitivity to batch sizes, we designed a *decoupled BN update scheme* that adapts the BN statistics and BN parameters *asynchronously* with different batch sizes, enabling effective model adaptation within a constrained memory budget.

We deploy EmbodiTTA on a Jetson Orin Nano, a portable computing platform commonly used in embodied systems, and evaluate it on four visual datasets spanning both object recognition and semantic segmentation tasks: object recognition with CIFAR10-C [4], ImageNet-C [4], CORE50 [5], and semantic segmentation with SHIFT [11]. The experimental results show that EmbodiTTA achieves higher TTA accuracy while reducing energy consumption by 47.2%, with comparable memory usage. Our comprehensive evaluation further highlights the robustness of EmbodiTTA, reinforcing its practical utility for real-world deployment under resource-constrained conditions.

Our contributions are summarized as follows:

- We introduce the concept of on-demand TTA for resource-constrained embodied visual systems. Our approach triggers adaptation only when significant domain shifts occur, thereby reducing adaptation frequency by up to 90% over C-TTA methods while achieving the highest overall TTA accuracy across nearly all evaluation settings.
- We present EmbodiTTA, a novel on-demand TTA framework for embodied devices. It comprises three core innovations: a lightweight domain shift detector, a source domain selection module, and a decoupled BN updating strategy.
- We implement EmbodiTTA on an embodied device and evaluate its performance across multiple datasets from various perspectives. Our results indicate that EmbodiTTA achieves higher TTA accuracy while reducing energy consumption by 47.2% and latency by $6.7\times$.

II. PRELIMINARY

In this section, we present a detailed comparison between conventional continual TTA and the proposed on-demand TTA, followed by three key insights that inspire the design of EmbodiTTA.

A. Continual TTA vs. On-demand TTA

Figure 2(a) depicts the process of existing TTA approaches, which perform two sequential steps for *each incoming batch of new data*: an *adaptation* step that updates certain layers (e.g., BN layers) by minimizing the entropy loss to align with the batch distribution, followed by an *inference* step that produces predictions with the adapted model. This paradigm, referred to as **continual TTA** (C-TTA), although achieving optimal performance, is impractical for real-world deployment: adapting on every batch before inference incurs substantial latency and energy overhead even when the domain shift between consecutive batches is trivial.

Instead, a more practical TTA paradigm is to adapt the model only when a significant domain shift occurs and the current model suffers from an unacceptable performance degradation. As shown in Figure 2(b), we introduce a new concept termed **on-demand TTA** for practical and efficient model adaptation during inference. On-demand TTA continuously monitors each incoming batch for potential domain shifts. If no shift is detected (e.g., Batch 1 and Batch N), the model proceeds with regular inference. However, once a domain shift is identified (e.g., Batch 2), the current model undergoes an adaptation process and performs inference on incoming data until the next domain shift is detected. Considering that domain shifts typically occur less frequently in real-world scenarios [12], on-demand TTA has the potential to significantly improve the utilization efficiency of on-device resources (e.g., computation and energy) by reducing the adaptation frequency, without sacrificing much accuracy.

B. Challenges for On-demand TTA

Despite potential benefits, realizing on-demand TTA on embodied devices presents several additional challenges compared to C-TTA. **First (when to adapt)**, on-demand TTA triggers adaptation only when a substantial domain shift is detected and model performance drops. However, quantifying the performance drop without the labels is challenging. Additionally, since domain shift detection needs to be performed continuously on every input data, the detection process must be lightweight to conserve on-device resources. **Second (where to adapt from)**, unlike continual TTA, where the distribution captured by the current model and the distribution of the incoming data are often similar, on-demand TTA encounters a large distribution disparity when an adaptation is triggered. Therefore, it is unclear whether adapting from a largely deviated model will still be effective. **Third (how to adapt)**, existing C-TTA approaches rely on large batch sizes to accurately capture the distribution of new data, which consumes a significant amount of memory during adaptation (storing intermediate tensors during backpropagation). However, embodied devices generally have limited memory and need to process streaming data, making effective TTA with small batch sizes (even batch size = 1) challenging.

C. Key Insights

Before introducing our solutions, we first present some key insights that inspire our design.

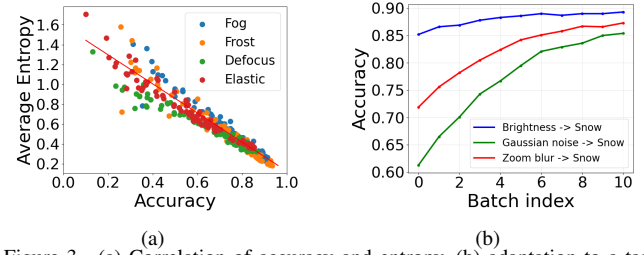


Figure 3. (a) Correlation of accuracy and entropy, (b) adaptation to a target domain from different source domains.

1) Approximating accuracy with entropy: The first objective in on-demand TTA is to detect the occurrence of domain shifts in a *lightweight* manner, as this needs to be performed continuously on all incoming data. Intuitively, one might consider tracking the accuracy drop during inference to identify such shifts. However, since the ground truth labels of the inference data are unavailable in practice, this method becomes infeasible. Inspired by entropy minimization [7] in unsupervised learning, which improves model performance by reducing the entropy of predictions, we hypothesize that *model performance is inversely correlated with entropy*. This insight leads us to explore using entropy as a potential metric to assess changes in model accuracy due to domain shifts.

To verify our hypothesis, we conducted an experiment using a pre-trained ResNet-50 model on CIFAR10-C, a common domain shift dataset comprising 15 different domains. We first adapted the source model to four selected domains using supervised learning to ensure effective adaptation. Then, we tested the adapted models on 75 subsets sampled from other domains, and we calculated the average accuracy and entropy in each subset. The results are shown in Figure 3(a), where each color represents the adapted model for each domain, and each scatter point denotes the accuracy and entropy of one subset. It is evident that entropy and accuracy are inversely correlated, with the relationship appearing nearly linear. **This inverse correlation between prediction entropy and model accuracy motivates us to approximate model performance by tracking the entropy of each input sample, without the need for ground-truth labels.**

2) Adaptation from closer domain results in higher effectiveness: C-TTA always adapts the model from the previous domain, which may not be effective in on-demand TTA due to substantial distribution shifts. Based on the key observation that different domains exhibit varying degrees of similarity (e.g., in weather conditions, foggy and frost seem closer, compared to foggy and sunny), we hypothesize that *the source domain (i.e., the domain before adaptation) can significantly impact the adaptation performance*. To test this, we adapted the model to the Snow domain (target domain) from three different source domains: Brightness, Zoom blur, and Gaussian noise. The accuracies of directly performing inference of the three source-domain models (i.e., models trained with the data from each domain only) on the Snow domain data were 85.2%, 71.9%, and 61.3%, respectively, which indicates that Brightness is the closest to Snow, followed by Zoom blur and Gaussian noise.

Figure 3(b) displays the adaptation accuracies with different

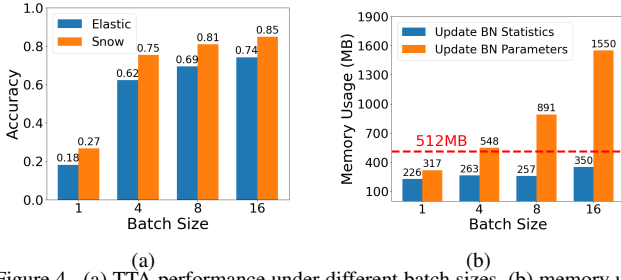


Figure 4. (a) TTA performance under different batch sizes, (b) memory usage of updating BN statistics and parameters.

numbers of batches. We can observe that adapting from a closer domain (i.e., Brightness) yields higher accuracy after adaptation, suggesting that the source domain indeed affects the adaptation performance. Moreover, starting from a closer domain leads to faster convergence: adapting from Brightness to Snow reaches a stable accuracy within four batches, whereas starting from Zoom Blur requires more than ten batches. Overall, these results suggest that **using a more similar source domain as the starting point provides a better initialization for adaptation, improving both the final adapted accuracy and the convergence speed.**

3) *Effectively updating BN statistics requires a large batch size yet incurs low memory usage:* Most existing TTA approaches only adapt the BN layers rather than the entire model, as it is sufficient to align the model with distribution shifts. However, these approaches rely on a large batch size (e.g., 64 in Tent [7] and EATA [8]) to capture the new domain's distribution. Figure 4(a) validates this by exemplifying the adaptation performance of EATA on the Elastic and Snow domain under different batch sizes, where smaller batch sizes yield poorer performance. On the other hand, larger batch sizes also demand significantly more memory to store intermediate activations and gradients for back-propagation, posing a challenge for memory-constrained embodied devices. For example, as shown in Figure 4(b), with a batch size of 4, the memory requirement for updating BN parameters of ResNet-50 can easily exceed the available memory space of some small embodied devices such as Raspberry Pi Zero 2W with only 512MB DRAM.

To address this challenge, we delved into the detailed operation of BN layers during adaptation. Our detailed analysis reveals that *BN statistics and BN affine parameters exhibit different sensitivities to batch size and impose distinct memory overheads*. We further analyzed the memory usage of updating BN statistics and BN parameters under different batch sizes in ResNet-50. As shown in Figure 4(b), increasing the batch size results in significantly higher memory consumption when updating BN affine parameters, as this process requires backpropagation. In contrast, the memory usage for updating BN statistics remains relatively stable, since it only involves the forward pass. **This finding reveals that the memory overhead associated with large batch size primarily stems from updating BN parameters. Conversely, updating BN statistics, which is crucial for capturing data distributions, can be performed using large batch sizes without incurring substantial memory overhead.**

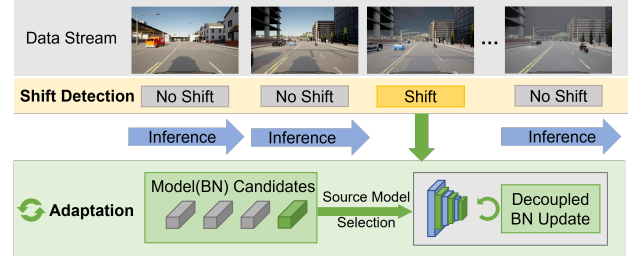


Figure 5. The overview of EmbodiTTA.

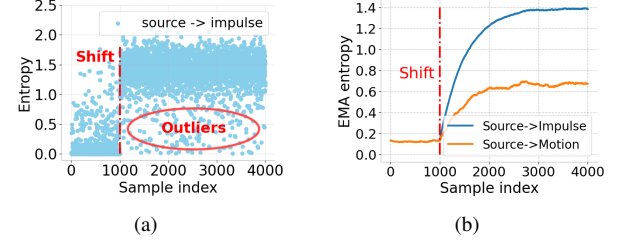


Figure 6. Sample-wise domain shift detection using (a) entropy and (b) the proposed EMA entropy.

III. EMBODITTA DESIGN

Based on the above insights, we propose EmbodiTTA, an on-demand TTA framework for resource-constrained embodied devices. As shown in Figure 5, EmbodiTTA comprises two modules: a lightweight *domain shift detector* (Section III-A) that monitors incoming data and triggers adaptation only when needed, and a *model adaptation* module that selects the closest source domain from a candidate pool (Section III-B) before applying a decoupled BN update (Section III-C) to align the model with the new domain under memory constraints.

A. Domain Shift Detection

1) *EMA entropy calculation:* In Section II-C1, we presented the inverse correlation between model accuracy and average entropy. However, this correlation is demonstrated on a subset of samples (*subset-wise*) and does not consistently hold in real-world scenarios where data is processed in a streaming manner. In such cases, DNNs operate on each individual sample, resulting in significant variations in *sample-wise* entropy. Specifically, Figure 6(a) illustrates the sample-wise entropy (denoted by each scatter) when the streaming samples shift from one domain (Source) to another domain (Impulse Noise). It is evident that while most samples in Impulse domain exhibit significantly higher entropy levels, some samples still present low entropy, hindering the direct detection of domain shifts using sample-wise entropy.

To address this problem, we introduce an exponential moving average (EMA) strategy to smooth the sample-wise entropy by incorporating historical entropy and reducing the influence of a single sample. The formula for calculating the EMA entropy is as follows:

$$E_t = m \cdot E_{t-1} + (1 - m) \cdot x_t. \quad (1)$$

where E_t represents the EMA entropy at time t , m denotes the momentum factor (with a value between 0 and 1), and x_t is the entropy value of the current input sample at time t that can be directly obtained from the inference process. The

momentum m influences the stability of the EMA entropy and its sensitivity to domain shifts. A higher momentum value causes the current entropy to contribute minimally to the EMA entropy, resulting in a more stable curve but reducing sensitivity to domain shifts. In this paper, we empirically set the momentum to 0.995 to balance this trade-off. Figure 6(b) illustrates the EMA entropy curves for two adaptations, which clearly indicate the effectiveness of EMA entropy in detecting domain shifts.

2) *Shift determination*: After each adaptation, EmbodiTTA records the EMA entropy over the next few samples (e.g., 100) as the entropy baseline (EMA_{base}), which reflects the current model's capability on the adapted domain data. With the incoming data stream, EmbodiTTA calculates the sample-wise EMA entropy directly from the model inference outputs, referred to as EMA_{sample} . Notably, the extra computation from entropy to EMA entropy involves only simple multiplication and addition and therefore the proposed method is very lightweight. Once $EMA_{sample} - EMA_{base}$ exceeds a threshold (EMA_{thr}), an adaptation is triggered. The threshold (EMA_{thr}) controls the trade-off between adaptation frequency and energy efficiency: a higher threshold tolerates more drift before triggering adaptation (saving energy), while a lower threshold adapts more readily for better accuracy at the cost of higher energy consumption. We provide more discussion of the detection sensitivity in Section V-B1.

To avoid manually selecting an absolute threshold for each dataset, we determine the threshold based on the historical distribution of EMA entropy under a stable data distribution. Specifically, we first compute the EMA entropy over a set of stable samples using the same momentum setting as during deployment. Let σ_E denote the standard deviation of the resulting EMA entropy sequence. We then define the detection threshold as

$$EMA_{thr} = k\sigma_E, \quad (2)$$

where k is a dataset-independent scaling factor. Accordingly, a domain shift is detected when

$$EMA_{sample} - EMA_{base} > k\sigma_E. \quad (3)$$

This formulation scales the detection threshold according to the natural fluctuation of EMA entropy under a stable distribution, avoiding the need to manually specify different absolute thresholds for different datasets. In our experiments, we use a unified setting of $k = 5$ across all datasets. The historical samples are used only to estimate σ_E , and no ground-truth labels are required for threshold selection.

B. Source Domain Selection

As noted in Section II-C2, adapting from a closer domain can enhance both the accuracy and the convergence speed of adaptation. This observation motivates us to select the domain most similar to the new domain from a candidate pool before adaptation, rather than directly adapting from the last domain, referred to as source domain selection¹. This process necessitates two essential steps: 1) constructing a pool

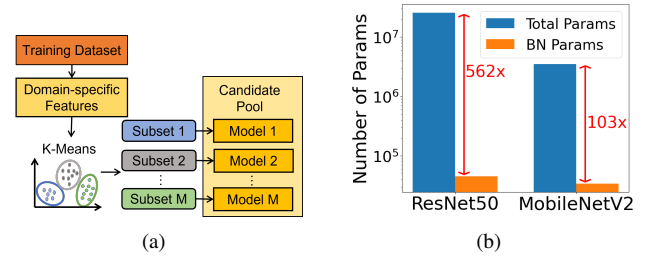


Figure 7. (a) Candidate domain construction, (b) storage comparison of saving only BN layers and the full model.

with enough candidates to ensure effectiveness from the start (Section III-B1) and during the runtime (Section III-B1), and 2) measuring the domain similarity to identify the candidate most closely aligned with the new domain (Section III-B2).

1) *Candidate pool construction*: Considering that a domain essentially reflects the distribution of a specific set of data, the process of creating a domain candidate pool can be converted as generating multiple sets of data with diverse distributions. Additionally, training datasets typically contain dispersed distributions due to the varied data sources and collection conditions. Therefore, to construct the candidate pool, we propose to split the training dataset into multiple subsets and adapt the pre-trained model on each subset.

Figure 7(a) depicts the detailed process². First, we extract features that can represent the domain characteristics (i.e., domain features) for each training sample. Specifically, given that BN statistics mainly capture data distribution, we feed each sample through the pre-trained model and utilize the output BN mean from the second BN layer as the domain features. We select the BN mean instead of the BN variance because the mean provides a stronger indication of domain shifts, while the variance primarily reflects distribution stability [13]. Shallow BN layers are preferred because they capture more domain-specific features, while deeper layers tend to focus on task-specific knowledge [13]. Meanwhile, we avoid using the first BN layer as it processes very primitive features immediately after the initial convolution, which are less informative for domain characterization. Second, based on these domain features, we cluster the training samples into m subsets using the K-Means algorithm [14], with each cluster representing a domain. Third, we adapt the pre-trained model on each subset by freezing all the layers except for the BN layers (same as conventional TTA), resulting in m domain candidates in the pool by saving the BN layers. Unlike TTA, this adaptation is supervised using known training labels to ensure accurate BN updates.

Alternatively, when source data is unavailable, EmbodiTTA can progressively build the candidate pool at runtime: starting from the source model as the initial candidate, each newly adapted model is stored (saving only its BN parameters) as an additional candidate for future domains. We evaluate this progressive construction scheme in Section V-B2, and the two strategies can be combined to build a stronger candidate pool.

²The candidate pool is constructed during model training, prior to online adaptation. This process does not require access to the training dataset during TTA.

¹Here the source domain refers to the domain before each adaptation.

Overhead of storing domain candidates: As TTA only adapts BN layers, storing the candidate models requires only storing the BN layers. To understand the storage overhead, we measured the storage requirements for saving just the BN layers compared to the entire model. Figure 7(b) demonstrates that BN parameters account for only 1/562 and 1/103 of the total parameters (translating to 480 KB and 210 KB) in ResNet-50 and MobileNetV2, respectively. This finding suggests that the overhead of storing multiple domain candidates (e.g., 100) is negligible in terms of memory consumption, promoting the feasibility of the proposed domain selection strategy.

2) *Similar candidate selection:* After constructing a candidate pool, the next step is to select the candidate domain most similar to the incoming target domain. This similarity estimation relies on accurately representing the new domain using BN statistics extracted via the source model. However, the source model typically performs poorly on shifted domains due to BN mismatches, which degrades the quality of extracted features (see Section II-C3).

Specifically, we cache N samples from the new domain (e.g., $N = 128$) and process them in small batches (e.g., batch size 16) through the source model to extract domain features. Before extraction, we update the BN statistics via a forward pass to align the model with the batch distribution³. Then, for each batch B_k , we compute the BN mean from the second BN layer and average over the K cached batches to obtain the final domain representation:

$$\mu_{\text{domain}} = \frac{1}{K} \sum_{k=1}^K \left(\frac{1}{|B_k|} \sum_{i \in B_k} \phi(x_i) \right), \quad (4)$$

where $\phi(x_i)$ denotes the intermediate output of sample i in batch B_k after the second BN layer, which we choose because it exhibits the highest correlation with domain similarity, as evaluated in Section V-B2.

To identify the most similar candidate model from the pool, we compute the L2 distance between μ_{domain} and each candidate's stored BN mean μ_c , and select the closest one:

$$c^* = \arg \min_c \|\mu_{\text{domain}} - \mu_c\|_2. \quad (5)$$

Here, c^* denotes the candidate whose BN statistics are most similar to the current domain representation. This selected candidate serves as the initialization point for the subsequent adaptation process. We further evaluate the effectiveness of this selection strategy in Section V-B2.

C. Decoupled BN Update

While similar candidate selection improves the performance of on-demand TTA, the challenge related to small batch sizes on resource-constrained embodied devices still persists. Recall that each BN layer contains two types of quantities: *BN statistics* (running mean/variance for normalization) and *BN parameters* (learnable affine scale γ and shift β); updating statistics requires only a forward pass and is sensitive to batch

size, whereas updating parameters via backpropagation is more robust to small batches [7], [8], [9], [10]. Inspired by the findings in Section II-C3, we propose a *decoupled BN update* strategy: BN statistics are first updated with a larger batch size during a forward pass, followed by BN parameter fine-tuning with a small batch size during backpropagation.

1) *BN statistics update:* Having cached N samples for similar model selection, we efficiently reuse these samples to adapt the BN layers. Specifically, the samples are also split into batches of size 16. But unlike similar domain selection, which updates BN statistics on the previous poor model by averaging over batches, here the BN statistics are updated on the selected candidate model in a batch-by-batch manner. In detail, we employ an Exponential Moving Average (EMA) approach to integrate the BN statistics of the current batch with historical BN statistics, defined as follows:

$$S_k = (1 - m) \cdot S_{k-1} + m \cdot B_k \quad (6)$$

where S_k represents the integrated BN statistics at batch k , m is the momentum factor, S_{k-1} is the BN statistics integrated from the previous batch, and B_k represents the BN statistics of the current batch. Particularly, S_0 is the BN statistics of the selected candidate model. The proposed EMA-based updating scheme allows us to seamlessly integrate the similar source model with the new domain samples, ensuring an accurate and robust alignment of both source and new domain distributions.

2) *BN parameters update:* After updating the BN statistics, the selected model already captures the distribution of the new data, but the BN parameters still need to be fine-tuned accordingly through back-propagation. To fit into the limited on-device memory (e.g., 512MB as plotted in Figure 4(b)), we aim to update the BN parameters with a small batch size (e.g., 1). However, back-propagation using a single sample is challenging due to the inherent instability of unsupervised learning [9]. To achieve stable fine-tuning, following EATA-C [15] and SAR [9], we filter out high-entropy samples when calculating the entropy loss, since such low-confidence predictions introduce noisy gradients that destabilize the unsupervised adaptation.

Therefore, the overall loss function is defined as:

$$\mathcal{L}_{\text{ent}} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(- \sum_{c=1}^C p_{i,c} \log p_{i,c} \right), \mathcal{S} = \{i \mid H(p_i) < \tau\}, \quad (7)$$

In Equation (7), $(-\sum_{c=1}^C p_{i,c} \log p_{i,c})$ is the prediction entropy. We compute the entropy-minimization loss only over the reliable subset $\mathcal{S}$, defined as samples whose entropy falls below a threshold τ . Following EATA-C [15] and SAR [9], we set $\tau = 0.4 \log(C)$, where C is the number of task classes. For the cached samples used in adaptation, the predictions produced during the forward pass for computing the BN-parameter adaptation loss are directly recorded as their final outputs. Therefore, no additional inference pass is performed after the parameter update.

IV. EVALUATION SETUP

Device. We implement EmbodiTTA on Jetson Orin Nano (8GB RAM) running PyTorch 2.0 on Ubuntu 20.04. Classi-

³The process only requires updating BN statistics via a forward pass without backpropagation, resulting in low peak memory consumption, as illustrated in Figure 4(b).

fication at batch sizes 1 and 16 is evaluated on Jetson Orin Nano whenever the method fits within the device memory; otherwise, the corresponding result is measured on the server. Batch size 64 is evaluated on the server for all methods.

Datasets. We evaluate on four datasets. **CIFAR10-C** and **ImageNet-C** [4] are corrupted variants of CIFAR10 [16] and ImageNet [17] with 15 corruption types at severity level 5. **CORE50** [5] is a streaming object recognition dataset collected across 11 sessions (8 indoor, 3 outdoor); we train on indoor sessions and run TTA on the three outdoor sessions as unseen target domains. **SHIFT** [11] is an autonomous driving segmentation dataset with day→night, clear→foggy, and clear→rainy shifts; we follow [11] and resize images to 640×400 to reduce on-device overhead.

Baselines. We compare against representative TTA methods: **SAR** [9] replaces BN with group normalization for batch-size robustness (using a TIMM ResNet-50-GN [18]); **MECTA** [10] reduces memory via selective parameter updates through a redesigned normalization layer; **ROID** [19] uses certainty/diversity-based weighting with continual weight averaging; **L-TTA** [20] adapts only lightweight stem layers to minimize updated tensors; **SURGEON** [21] dynamically prunes activations for a controllable accuracy–memory trade-off; and **EATA-C** [15] extends EATA [8] with uncertainty-aware sample filtering and calibration; RoTTA [22] maintains a category-balanced memory bank to improve continual adaptation under dynamic test streams. We include RoTTA as a representative memory-bank-based TTA method to provide a direct comparison with cache-aware adaptation.

Adaptation Details. For object recognition we use ResNet-50 [23]; for segmentation, DeepLabV3 [24] with a ResNet-50 backbone (metric: mIoU). Baselines use their default settings. Unless otherwise specified, the reported batch size refers to both the inference batch size and the batch size used for gradient-based adaptation. EmbodiTTA uses a larger batch for the decoupled BN-statistics update; however, this step is forward-only and consumes substantially less memory than backpropagation. As shown in Fig. 4(b), its memory usage remains below that of the gradient-based update even with a larger batch size. Hence, under the same peak-memory budget, the backpropagation batch size remains the dominant memory constraint and provides a fair basis for comparison. Cached samples are used for domain selection and BN-statistics estimation rather than increasing the backpropagation batch. For EmbodiTTA, we use 128/512/512 adaptation samples on CIFAR10-C/ImageNet-C/CORE50 at batch size 1, and 512 samples for batch sizes 16 and 64. Learning rates are 1×10^{-4} (CIFAR10-C, SHIFT) and 1×10^{-5} (ImageNet-C). For shift detection, we determine the threshold as $\text{EMA}_{\text{thr}} = k\sigma_E$, where σ_E is estimated from the EMA entropy under a stable historical distribution. We use a unified setting of $k = 5$ across all datasets without using target labels.

V. EVALUATION RESULTS

A. Adaptation Performance

1) **CIFAR10-C and ImageNet-C:** Table I shows that EmbodiTTA achieves significant accuracy gains across all

batch sizes. Notably, it achieves strong batch-size-1 performance among BN-based methods. It is critical for memory-constrained embodied devices—while SAR (GN-based) underperforms at larger batches. Memory-wise, EmbodiTTA achieves peak GPU usage comparable to EATA-C and SAR. Although MECTA and L-TTA can reduce memory at the same batch size through sparse or optimized backpropagation, their accuracy degrades substantially at batch size 1. To reach accuracy comparable to EmbodiTTA at batch size 1 on ImageNet-C (33.8%, 414 MB), MECTA must increase the batch size to 16, where it consumes 1231 MB. This highlights that EmbodiTTA achieves a better accuracy–memory trade-off under small-batch embodied deployment.⁴ Compared with RoTTA, a representative memory-bank-based TTA method, EmbodiTTA achieves a substantially better accuracy–memory trade-off. RoTTA maintains historical target samples and uses the memory bank for adaptation, which leads to considerable memory overhead as the adaptation batch size increases. In contrast, the cached samples in EmbodiTTA are mainly used to obtain more reliable target-domain statistics and update BN statistics through forward-only operations, while the gradient-based update is still performed with a small backpropagation batch. Consequently, EmbodiTTA can exploit information from multiple observed samples without incurring the memory cost of backpropagating through a large cached batch.

2) **CORE50:** Table II shows that EmbodiTTA achieves the best performance across all batch-size settings, reaching 83.7% at batch size 1 and substantially outperforming the strongest baseline, SAR (76.8%). Many baselines collapse at batch size 1 (e.g., ROID: 6.2%, L-TTA: 2.2%), highlighting the gap between conventional TTA evaluation and embodied deployment where small batches are often unavoidable. EmbodiTTA remains stable across all batch sizes.

3) **SHIFT:** Table III shows that EmbodiTTA consistently outperforms all compared baselines across all shift types (avg. mIoU 25.1% vs. 24.0% for the next best, SURGEON).⁵ Notably, SAR—which achieves competitive recognition accuracy—performs worst on segmentation (avg. mIoU 11.8%), underscoring the task-sensitivity of normalization design choices.

4) **Latency and Energy:** As shown in Table IV, EmbodiTTA processes 10,000 samples in 239 s, $6.7\times$ faster than MECTA (latency baseline). The speedup grows with sequence length because adaptation fires only on detected shifts, eliminating the per-batch overhead of C-TTA. Figure 8 shows that EmbodiTTA saves 47.2% energy over EATA-C at 10,000 frames, with consistent gains at batch size 16.⁶ For real-world shifts (e.g., weather, day/night transitions) that persist for hours, the savings are even larger.

In summary, EmbodiTTA outperforms nearly all baselines across metrics and offers the strongest trade-off among accu-

⁴Memory refers to peak GPU usage across the entire process including model loading, backpropagation, and runtime operations.

⁵ROID and L-TTA are not included in the SHIFT evaluation as their normalization designs are tightly coupled to classification heads and do not support semantic segmentation.

⁶ROID cannot run at batch size 16 on device due to memory constraints.

Table I

COMPARISON OF ACCURACY (%) ON CIFAR10-C AND IMAGENET-C USING RESNET-50. MEMORY IS MEASURED ON JETSON ORIN NANO WHENEVER THE METHOD FITS ON THE DEVICE; OTHERWISE, IT IS MEASURED ON A SERVER. BATCH SIZE 64 IS MEASURED ON A SERVER FOR ALL METHODS DUE TO JETSON MEMORY LIMITATIONS.

Method	Batch Size = 1			Batch Size = 16			Batch Size = 64		
	Avg. Acc.(%)		Memory (MB)	Avg. Acc.(%)		Memory (MB)	Avg. Acc.(%)		Memory (MB)
	CIFAR10-C	ImageNet-C		CIFAR10-C	ImageNet-C		CIFAR10-C	ImageNet-C	
Source	59.5	26.9	242	59.5	26.9	358	59.5	26.9	809
SAR	68.3	35.1	429	70.4	35.2	1723	69.2	37.3	5780
MECTA	65.0	12.0	378	81.3	33.2	1231	81.3	33.7	3836
ROID	10.1	0.4	698	79.5	32.5	2850	82.8	37.8	9840
L-TTA	10.1	0.1	449	76.8	26.6	1250	81.2	34.5	3580
SURGEON	10.0	0.4	454	74.5	9.97	1750	82.0	36.3	5882
EATA-C	23.7	1.0	506	77.4	31.9	1728	81.3	38.6	6228
RoTTA	66.2	32.8	2829	82.9	33.9	11328	81.9	34.9	22588
Ours	78.0	33.8	414	83.0	37.4	1632	84.9	40.4	5653

Table II
COMPARISON OF ACCURACY (%) ON CORE50 DATASET USING RESNET-50.

Method	Batch size = 1	Batch size = 16	Batch size = 64
Source	69.9	69.9	69.9
SAR	76.8	75.9	74.6
MECTA	62.7	83.4	84.2
ROID	6.2	81.3	84.3
L-TTA	2.2	83.0	83.3
SURGEON	2.9	79.5	80.4
EATA-C	6.7	73.4	78.0
Ours	83.7	84.0	84.5

Table III
COMPARISON OF ADAPTATION MIOU (%) ON SHIFT USING DEEPLABV3.

Method	SHIFT types			Avg.
	Day→Night	Clear→Foggy	Clear→Rainy	
Source	27.3	17.7	11.0	18.7
SAR	23.6	7.6	4.2	11.8
MECTA	24.2	20.6	15.4	20.1
SURGEON	30.1	23.6	18.2	24.0
EATA-C	24.7	21.5	16.7	20.9
Ours	31.1	25.2	19.1	25.1

racy, memory footprint, and energy consumption, making TTA a practical, deployable solution for embodied devices.

B. Per-component Evaluation

1) *Analysis of domain shift detection:* Figure 9 illustrates the EMA entropy over CIFAR10-C streams, with detected adaptation triggers marked by red dotted lines. EmbodiTTA detects 13 of 15 domain shifts; the two missed transitions (Gaussian Noise → Shot Noise, Motion Blur → Zoom Blur) caused no accuracy degradation—performance actually improved by 1.4% and 5.2% respectively, confirming that

Table IV
LATENCY FOR PROCESSING DOMAIN DATA SEQUENCES OF VARYING LENGTHS UNDER BATCH SIZE OF 1 ON CIFAR10-C.

Lengths	1000	3000	5000	7000	9000	10000
SAR	138 (1.2×)	414	690	965	1241	1379 (1.2×)
MECTA	160 (1.0×)	480	800	1120	1440	1600 (1.0×)
ROID	75 (2.1×)	226	376	526	676	749 (2.1×)
L-TTA	50 (3.2×)	143	240	333	424	477 (3.4×)
SURGEON	78 (2.1×)	227	376	531	678	770 (2.1×)
EATA-C	47 (3.4×)	136	227	317	408	453 (3.5×)
Ours	47 (3.4×)	90	132	175	218	239 (6.7×)

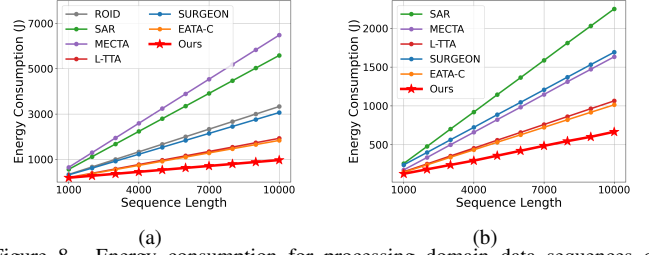


Figure 8. Energy consumption for processing domain data sequences of varying lengths under batch size = (a) 1 and (b) 16.

Table V
IMPACT OF DIFFERENT DOMAIN SHIFT ORDERS (S1–S10). EACH SEQUENCE IS A RANDOM SHUFFLE OF THE 15 DOMAINS.

Sequences	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	Mean ± Std.
Triggered	15	13	15	14	13	14	15	13	13	14	13.9 ± 0.8
Missed	1	2	0	1	2	1	0	2	2	1	1.2 ± 0.7
False	1	0	0	0	0	0	0	0	0	0	0.1 ± 0.3
Delay	301	569	262	837	322	279	477	531	488	523	459 ± 167
Avg. acc.	79.9	79.3	80.5	79.5	79.5	80.2	80.2	78.7	77.7	79.7	79.5 ± 0.8

skipping adaptation is the correct behavior when the shift is benign. For detected shifts, 7 of 13 are identified within 100 frames. To evaluate robustness to domain ordering, we test 10 randomly shuffled domain sequences: the detector identifies 13.9 of 15 shifts on average, with only 0.1 false triggers per sequence, while TTA accuracy remains stable at 79.5%.

To further evaluate the robustness of the proposed shift detector to different domain orders, we construct 10 randomized domain sequences on CIFAR10-C. Specifically, we randomly shuffle the 15 corruption domains to generate 10 sequences (S1–S10). Each sequence contains 150,000 test samples, with domain transitions occurring every 10,000 samples, and is evaluated under the same batch-size-1 setting as the main experiments. Table V reports the number of detected shifts, missed shifts, false triggers, detection delay, and the resulting TTA accuracy. Across the 10 randomized sequences, the detector produces an average of 13.9 detection triggers, with only 1.2 missed shifts and 0.1 false triggers per sequence. The average detection delay is 459 samples. Moreover, the resulting TTA accuracy remains highly stable across different domain orders, averaging 79.5% with a standard deviation of only 0.8%. These results demonstrate that the proposed detector is robust to variations in domain ordering and that variations in detection timing do not substantially affect down-

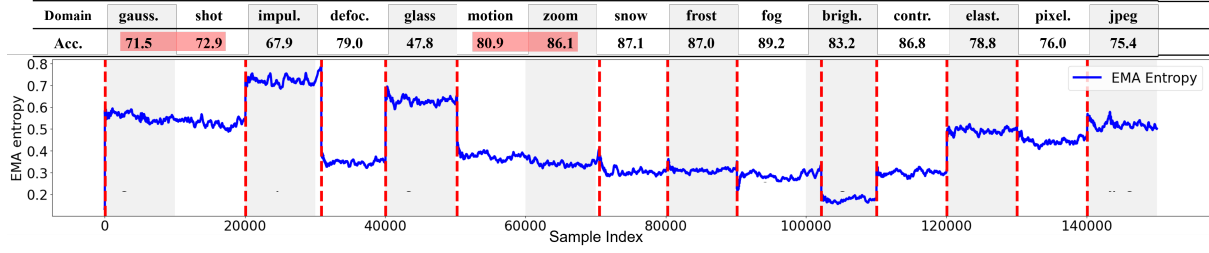


Figure 9. EMA entropy change along the data stream on CIFAR10-C. Domain transitions occur every 10,000 samples and are highlighted in alternating background (white-gray) color. Red dotted lines indicate detected shifts. The table above shows the corresponding accuracy on each domain.

Table VI
THRESHOLD SENSITIVITY
ON CIFAR10-C.

EMA _{thr}	Triggers	Accuracy (%)
0.03	19	80.1
0.04	16	78.8
0.05	15	78.5
0.06	13	78.0
0.07	12	75.5

Table VII
ON-DEMAND ADAPTATION ABLATION ON
CIFAR10-C.

Method	Continual	Fixed-budget	On-demand
Avg. Acc. (%)	78.2	77.8	77.0
Adapt. Samples	150K	1.92K	1.66K
Workload Red. (%)	—	98.7	98.9

Table VIII
EVALUATION OF SOURCE MODEL SELECTION ON CIFAR10-C AND
IMAGENET-C.

Adapt from.	Batch Size = 1		Batch Size = 16		Batch Size = 64	
	CIFAR10-C	ImageNet-C	CIFAR10-C	ImageNet-C	CIFAR10-C	ImageNet-C
Prev-Domain	79.5	27.8	84.4	31.4	86.0	33.4
Initial-Pool	81.0	34.6	85.7	36.7	86.1	39.1
Progressive-Pool	81.7	34.5	85.1	37.5	86.0	37.7

stream adaptation performance.

Threshold sensitivity. We further evaluate the sensitivity of the shift detector to different detection thresholds on CIFAR10-C. As shown in Table VI, decreasing EMA_{thr} from 0.07 to 0.03 progressively increases the number of adaptation triggers from 12 to 19, while the TTA accuracy increases from 75.5% to 80.1%. This trend reflects the expected accuracy–efficiency trade-off of on-demand adaptation: a smaller threshold makes the detector more sensitive and triggers adaptation more frequently, whereas a larger threshold avoids unnecessary adaptation but may tolerate more performance degradation before adaptation is activated. Notably, the accuracy remains relatively stable over a broad range of thresholds from 0.03 to 0.06, while the number of triggers decreases substantially. For CIFAR10-C, the standard deviation of the EMA entropy under the stable source distribution is $\sigma_E = 0.012$, and the proposed setting $k = 5$ corresponds to $EMA_{thr} = 0.06$, which is also the default threshold used in our experiments. These results demonstrate that the detector does not rely on a narrowly tuned threshold and support $k = 5$ as a practical operating point that balances adaptation accuracy and frequency.

Effect of on-demand adaptation. To isolate the contribution of the on-demand mechanism from the other components of EmbodiTTA, we remove both source-domain selection and the decoupled BN update, and compare three adaptation strategies on CIFAR10-C. Continual adaptation updates the model throughout each target domain. Fixed-budget adaptation instead uses only the first 128 samples of each domain for adaptation, serving as a control that reduces the adaptation budget without using the proposed shift detector. On-demand adaptation uses the same per-update budget of 128 samples, but performs adaptation only when a domain shift is detected. As shown in Table VII, continual, fixed-budget, and on-demand adaptation achieve average accuracies of 78.2%, 77.8%, and 77.0%, respectively. The fixed-budget strategy already achieves accuracy close to continual adaptation while reducing the number of adaptation samples from 150K to 1.92K, showing that continuously updating the model provides

only marginal additional benefit. The on-demand mechanism further reduces the adaptation workload to 1.66K samples by skipping unnecessary updates, corresponding to a 98.9% reduction relative to continual adaptation. These results demonstrate that the proposed detector further improves adaptation efficiency beyond simply imposing a fixed adaptation budget, which is particularly beneficial for resource-constrained embodied systems.

2) Analysis of source domain selection: We use BN statistics from the second BN layer as the domain representation for candidate selection, following the design choice justified in Section III-B2. Table VIII compares adapting from the previous domain (*Prev-Domain*), a pool built from source data (*Initial-Pool*), and one built progressively from runtime history (*Progressive-Pool*). Domain selection consistently outperforms *Prev-Domain*, yielding up to 6.8% accuracy gains on ImageNet-C. *Progressive-Pool* matches *Initial-Pool* closely, confirming effectiveness even without prior source data access.

It is worth noting that *Progressive-Pool* is not expected to monotonically converge to *Initial-Pool*, since the two candidate pools are constructed from different domain sources. *Initial-Pool* captures the domain diversity available in the source dataset, whereas *Progressive-Pool* reflects historical target domains encountered during deployment. Therefore, their relative performance depends on which pool provides better coverage of the current target distribution. This is reflected in Table VIII: *Progressive-Pool* performs better in some settings, while *Initial-Pool* is advantageous in others. The two strategies thus provide complementary options for deployments with and without prior access to source data, and can also be combined when both sources of candidate domains are available.

3) Analysis of decoupled BN update: To assess the impact of adaptation sample count, we disable the shift detection module and adapt the model using varying numbers of samples (64 to 2048) from each domain in ImageNet-C, averaging accuracy across all 15 domains. Accuracy steadily improves and plateaus beyond 1024 samples, indicating that the new domain distribution is fully captured at that point. The decoupled update also reduces memory substantially: by separating

Table IX
ADAPTATION ACCURACY (%) ON IMAGENET-C USING MOBILENETV2
AND MOBILEViT.

Method	MobileNetV2			MobileViT		
	BS = 1	BS = 16	BS = 64	BS = 1	BS = 16	BS = 64
SAR	0.1	21.4	25.7	0.1	30.6	32.9
MECTA	0.2	0.6	0.7	0.2	0.2	0.2
ROID	0.2	21.5	25.4	0.2	30.0	33.1
SURGEON	0.2	1.38	23.5	0.2	2.51	28.4
EATA-C	0.1	21.8	26.2	0.9	27.3	32.5
Ours	14.8	22.5	26.8	22.4	32.8	33.3

BN statistics (forward-pass only, large batch) from BN parameters (backpropagation, small batch), EmbodiTTA reduces peak memory by $\sim 66\%$ compared to MECTA (414 MB vs. 1231 MB) at comparable accuracy.

C. Generalizability Across Model Architectures

EmbodiTTA consistently outperforms all baselines on MobileNetV2 and MobileViT (Table IX), and remains the only method effective at batch size 1 across both architectures, confirming that its gains generalize beyond ResNet-50.

VI. DISCUSSION

Accessing source data. In the TTA setting, direct access to the source domain dataset is typically restricted during adaptation. However, it remains unclear whether accessing the source data prior to adaptation, for purposes such as pre-computation or model customization, is permissible. For example, methods like EATA-C and L-TTA require access to source data before adaptation begins. In contrast, EmbodiTTA provides greater flexibility. When prior access to source data is available, it can be used to construct the candidate pool. If such access is not permitted, EmbodiTTA can instead employ a progressive strategy that builds the pool during runtime without requiring any source data.

Generalizing to other modalities. Beyond vision, embodied systems are typically equipped with diverse sensors (e.g., IMU, temperature, pressure, and light sensors) that produce continuous time-series streams and can also experience distribution shifts due to changes in the environment and sensor drift. Although this paper focuses on visual perception, the proposed on-demand TTA paradigm is broadly applicable to these modalities.

VII. RELATED WORK

A. Embodied Perception

Embodied systems need to perceive and act in open-world environments over long time horizons, where sensing conditions inevitably evolve with time, location, and operating context (e.g., illumination, weather, and backgrounds). Such domain shift is widely recognized as a key challenge for embodied AI [25], [26], [27]. To facilitate research in this setting, several benchmarks capture realistic long-term distribution changes, including CORE50 [5], EPIC-KITCHENS [28], and SHIFT [11], enabling systematic study of long-term robustness under evolving environments.

B. Continual Test-Time Adaptation

C-TTA [29], [6], [30] mitigates domain shift by adapting model parameters at test time using unsupervised objectives. Benz et al. [31] highlighted the critical role of batch normalization (BN) in enabling adaptation under distribution shifts. Building on this insight, Wang et al. [7] proposed TENT, which updates BN parameters via entropy minimization. SAR [9] replaces BN with Group Normalization (GN) to enhance stability. To improve robustness under streaming settings, SoTTA [32] maintains a sample-balanced cache for more stable adaptation. ROID [19] and CoTTA [33] are more memory-intensive, as they generate multiple augmented views and use a mean-teacher strategy for adaptation. Recent works have also focused on improving TTA efficiency. MECTA [10] replaces BN with a custom normalization layer to reduce memory usage. L-TTA [20] introduces lightweight stem layers at shallow stages and adapts only these stems to reduce memory overhead. SURGEON [21] minimizes memory overhead during backpropagation by updating only the most critical weights. EATA-C [15] leverages uncertainty estimates to filter unreliable samples, improving robustness during test-time adaptation.

Our work differs from existing research by introducing an on-demand TTA paradigm and a suite of techniques that improve adaptation effectiveness and efficiency under resource-constrained embodied deployment.

C. Domain Shift Detection

Domain shift detection is an essential part of EmbodiTTA, which monitors the distribution shift in the data stream to trigger the adaptation. Existing detection methods mainly utilize the martingale and statistics to measure the domain change. Luo [34] and Vovk [35] employ an auxiliary neural network to predict the martingale for each sample, serving as an indicator of domain shifts. However, they are memory-intensive because the auxiliary network (a dynamic CNN) must be continuously updated. Guy et al. [36] calculates a generalization bound for the source domain using the source dataset and identifies domain shifts by checking whether test samples exceed this established boundary. While this approach is less demanding in terms of memory, it is data-intensive, requiring substantial training data. Recently, Chakrabarty [37] and Niloy [13] proposed using the mean of features extracted from a batch of data to represent the domain of the batch and reset the model to the source when the domain gap is over the threshold to achieve reliable C-TTA. However, these feature-based methods rely heavily on large batch sizes, making them unsuitable for online data streams where data arrives sequentially and in smaller batches.

Our detection approach is both lightweight and effective, offering a significant advantage over other methods by being adaptable to any batch-size configuration.

VIII. CONCLUSION

This paper proposes a novel concept called on-demand TTA for resource-constrained embodied systems, which triggers adaptation only when a domain shift is detected. We introduce

EmbodiTTA, a framework designed to realize on-demand TTA for edge devices. EmbodiTTA comprises three key components: domain shift detection to monitor distribution shifts on the fly, source domain selection to optimize the efficacy of the source model for adaptation, and decoupled BN adaptation to update the model efficiently under limited memory constraints. The experimental results show that EmbodiTTA outperforms baselines across nearly all evaluation settings in accuracy, latency, and energy consumption while maintaining comparable memory overhead, which promotes its deployment on embodied devices in real-world scenarios.

REFERENCES

- [1] H. Liu, D. Guo, and A. Cangelosi, "Embodied intelligence: A synergy of morphology, action, perception and learning," *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [2] R. Geirhos, C. R. Temme, J. Rauber, H. H. Schütt, M. Bethge, and F. A. Wichmann, "Generalisation in humans and deep neural networks," *Advances in neural information processing systems*, vol. 31, 2018.
- [3] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do imagenet classifiers generalize to imagenet?" in *International conference on machine learning*. PMLR, 2019, pp. 5389–5400.
- [4] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," *arXiv preprint arXiv:1903.12261*, 2019.
- [5] V. Lomonaco and D. Maltoni, "Core50: a new dataset and benchmark for continuous object recognition," in *Proceedings of the 1st Annual Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, S. Levine, V. Vanhoucke, and K. Goldberg, Eds., vol. 78. PMLR, 13–15 Nov 2017, pp. 17–26. [Online]. Available: <https://proceedings.mlr.press/v78/lomonaco17a.html>
- [6] J. Liang, R. He, and T. Tan, "A comprehensive survey on test-time adaptation under distribution shifts," *International Journal of Computer Vision*, vol. 133, no. 1, pp. 31–64, 2025.
- [7] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," *arXiv preprint arXiv:2006.10726*, 2020.
- [8] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," in *International conference on machine learning*. PMLR, 2022, pp. 16 888–16 905.
- [9] S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan, "Towards stable test-time adaptation in dynamic wild world," *arXiv preprint arXiv:2302.12400*, 2023.
- [10] J. Hong, L. Lyu, J. Zhou, and M. Spranger, "Mecta: Memory-economic continual test-time model adaptation," in *2023 International Conference on Learning Representations*, 2023.
- [11] T. Sun, M. Segu, J. Postels, Y. Wang, L. Van Gool, B. Schiele, F. Tombari, and F. Yu, "Shift: a synthetic driving dataset for continuous multi-task domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 21 371–21 382.
- [12] D. Liu, J. Hou, S. Huang, J. Liu, Y. He, B. Zheng, J. Ning, and J. Zhang, "Lote-animal: A long time-span dataset for endangered animal behavior understanding," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 20 064–20 075.
- [13] F. F. Niloy, S. M. Ahmed, D. S. Raychaudhuri, S. Oymak, and A. K. Roy-Chowdhury, "Effective restoration of source knowledge in continual test time adaptation," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 2091–2100.
- [14] J. MacQueen *et al.*, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, vol. 1, no. 14. Oakland, CA, USA, 1967, pp. 281–297.
- [15] M. Tan, G. Chen, J. Wu, Y. Zhang, Y. Chen, P. Zhao, and S. Niu, "Uncertainty-calibrated test-time model adaptation without forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [16] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [17] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [18] R. Wightman, "Pytorch image models," <https://github.com/rwightman/pytorch-image-models>, 2019.
- [19] R. A. Marsden, M. Döbler, and B. Yang, "Universal test-time adaptation through weight ensembling, diversity weighting, and prior correction," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 2555–2565.
- [20] J. Shin and H. Kim, "L-tta: Lightweight test-time adaptation using a versatile stem layer," *Advances in Neural Information Processing Systems*, vol. 37, pp. 39 325–39 349, 2024.
- [21] K. Ma, J. Tang, B. Guo, F. Dang, S. Liu, Z. Zhu, L. Wu, C. Fang, Y.-C. Chen, Z. Yu *et al.*, "Surgeon: Memory-adaptive fully test-time adaptation via dynamic activation sparsity," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 30 514–30 523.
- [22] L. Yuan, B. Xie, and S. Li, "Robust test-time adaptation in dynamic scenarios," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 15 922–15 932.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [24] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.
- [25] L. Kunze, N. Hawes, T. Duckett, M. Hanheide, and T. Krajník, "Artificial intelligence for long-term robot autonomy: A survey," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4023–4030, 2018.
- [26] M. Wulfmeier, A. Bewley, and I. Posner, "Addressing appearance change in outdoor robotics with adversarial domain adaptation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1551–1558.
- [27] S. Siva and H. Zhang, "Robot perceptual adaptation to environment changes for long-term human teammate following," *The International Journal of Robotics Research*, vol. 41, no. 7, pp. 706–720, 2022.
- [28] D. Damen, H. Doughty, G. M. Farinella, S. Fidler, A. Furnari, E. Kazakos, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray, "The epic-kitchens dataset: Collection, challenges and baselines," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 43, no. 11, pp. 4125–4141, 2021.
- [29] Z. Wang, Y. Luo, L. Zheng, Z. Chen, S. Wang, and Z. Huang, "In search of lost online test-time adaptation: A survey," *International Journal of Computer Vision*, pp. 1–34, 2024.
- [30] Z. Xiao and C. G. Snoek, "Beyond model adaptation at test time: A survey," *arXiv preprint arXiv:2411.03687*, 2024.
- [31] P. Benz, C. Zhang, A. Karjauv, and I. S. Kweon, "Revisiting batch normalization for improving corruption robustness," in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 2021, pp. 494–503.
- [32] T. Gong, Y. Kim, T. Lee, S. Chottananurak, and S.-J. Lee, "Sotta: Robust test-time adaptation on noisy data streams," *Advances in Neural Information Processing Systems*, vol. 36, pp. 14 070–14 093, 2023.
- [33] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7201–7211.
- [34] R. Luo, R. Sinha, Y. Sun, A. Hindy, S. Zhao, S. Savarese, E. Schmerling, and M. Pavone, "Online distribution shift detection via recency prediction," *arXiv preprint arXiv:2211.09916*, 2022.
- [35] V. Vovk, I. Nourtdinov, and A. Gammerman, "Testing exchangeability on-line," in *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, 2003, pp. 768–775.
- [36] G. Bar Shalom, Y. Geifman, and R. El-Yaniv, "Window-based distribution shift detection for deep neural networks," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [37] G. Chakrabarty, M. Sreenivas, and S. Biswas, "A simple signal for domain shift," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3577–3584.